

Week 8

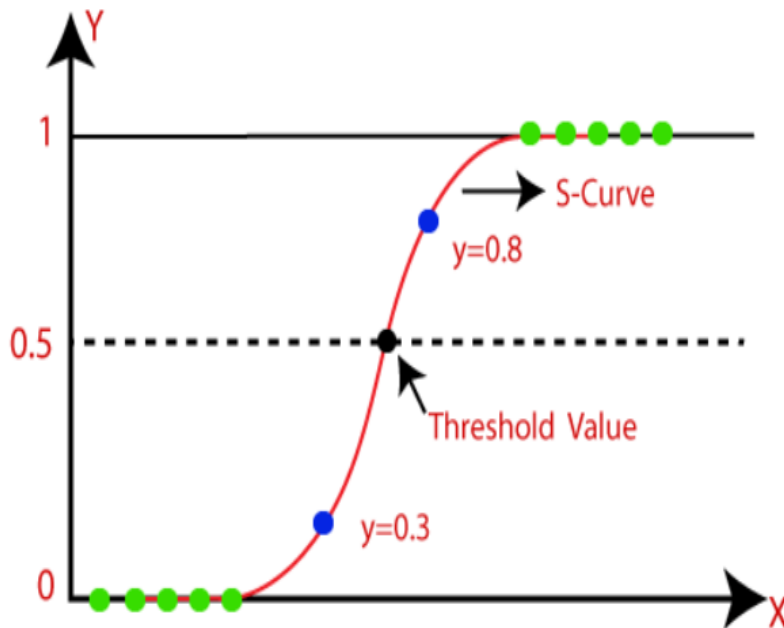
Logistic Regression

- Logistic regression is one of the most popular Machine Learning algorithms, which comes under the Supervised Learning technique. It is used for predicting the categorical dependent variable using a given set of independent variables.
- Logistic regression predicts the output of a categorical dependent variable. Therefore the outcome must be a categorical or discrete value. It can be either Yes or No, 0 or 1, true or False, etc. but instead of giving the exact value as 0 and 1, **it gives the probabilistic values which lie between 0 and 1.**
- Logistic Regression is much similar to the Linear Regression except that how they are used. Linear Regression is used for solving Regression problems, whereas **Logistic regression is used for solving the classification problems.**
- In Logistic regression, instead of fitting a regression line, we fit an "S" shaped logistic function, which predicts two maximum values (0 or 1).
- The curve from the logistic function indicates the likelihood of something such as whether the cells are cancerous or not, a mouse is obese or not based on its weight, etc.
- Logistic Regression is a significant machine learning algorithm because it has the ability to provide probabilities and classify new data using continuous and discrete datasets.

Logistic Function (Sigmoid Function):

- The sigmoid function is a mathematical function used to map the predicted values to probabilities.
- It maps any real value into another value within a range of 0 and 1.
- The value of the logistic regression must be between 0 and 1, which cannot go beyond this limit, so it forms a curve like the "S" form. The S-form curve is called the Sigmoid function or the logistic function.
- In logistic regression, we use the concept of the threshold value, which defines the probability of either 0 or 1. Such as values above the threshold value tends to 1, and a value below the threshold values tends to 0.
- Logistic Regression can be used to classify the observations using different types of data and can easily determine the most effective variables used for the classification.

- The below image is showing the logistic function:



Assumptions for Logistic Regression:

- The dependent variable must be categorical in nature.
- The independent variable should not have multi-collinearity.

Logistic Regression Equation:

The Logistic regression equation can be obtained from the Linear Regression equation. The mathematical steps to get Logistic Regression equations are given below:

- We know the equation of the straight line can be written as:

$$y = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \dots + b_nx_n$$

- In Logistic Regression y can be between 0 and 1 only, so for this let's divide the above equation by (1-y):

$$\frac{y}{1-y}; 0 \text{ for } y=0, \text{ and infinity for } y=1$$

- But we need range between $-\infty$ to $+\infty$, then take logarithm of the equation it will become:

$$\log \left[\frac{y}{1-y} \right] = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \dots + b_nx_n$$

The above equation is the final equation for Logistic Regression.

- *y is the probability of an event to happen which you are trying to predict*
- *x1, x2 and x3 are the independent variables which determine the occurrence of an event i.e. y*
- *b0 is the constant term which will be the probability of the event happening when no other factors are considered*
- *R.H.S represent the link function which will help us to determine a non-linear relation in a linear way wherein (y/1-y) is the odd ratio. Whenever the log of the odds ratio is found to be positive, the probability of success is always more than 50%.*

Type of Logistic Regression:

On the basis of the categories, Logistic Regression can be classified into three types:

- **Binomial:** In binomial Logistic regression, there can be only two possible types of the dependent variables, such as 0 or 1, Pass or Fail, etc.
- **Multinomial:** In multinomial Logistic regression, there can be 3 or more possible unordered types of the dependent variable, such as "cat", "dogs", or "sheep"
- **Ordinal:** In ordinal Logistic regression, there can be 3 or more possible ordered types of dependent variables, such as "low", "Medium", or "High"

Applications of logistic regression

When classification is needed on the dependent variable, this test is applicable. Below are some examples showing its application.

- A credit card company can apply logistic regression to categorize the people in two types; good credit and bad credit based on the characteristics such as annual salary, monthly credit card payments and number of defaults.
- A hospital can apply this test to classify patients into critical and non-critical categories based on selected health attributes.
- Insurance companies use logistic regression to predict the chances that the policy holder will die before the term of policy expires based on characteristics like gender, age and physical examination.
- A bank may use this test to predict the chances that a loan applicant will default on loan or not, based on past debts, past defaults and annual income.
- Marketers may use it in examining whether a customer will respond to a particular ad based on preference of customer and type of web portal customer is active.

- The case dataset here contains two series, series X and Y. Series X indicates the **CGPA (Cumulative Grade Points Average)** of ten students of a school. Series Y indicates whether these students gets admission in college, where 1= admit, and 0=no admit. The table below represents their values.

(CGPA) X	(Admission)Y
6	0
4	0
7.7	1
10	1
8.9	1
5.5	0
6.7	0
8.9	1
7.6	1
8.7	1

Table 1: Values for logistic regression case 1

- Here, logistic regression will help assess what level of **CGPA** leads to admission in college. This is called a ‘binary classification’ (either 1 or 0) problem. These types of cases need logistic regression. The aim here is to come up with the probability function that takes input X (**CGPA**) with outcome ‘what is the probability of getting admission’. Hence this test estimates a probability of each student based on its score. Furthermore, based on that probability, the analysis predicts if the student will get admission or not.

Build and Evaluate Logistic regression model in python

```
#IMPORT PACKAGES
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn import metrics
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score
import seaborn as sns
```

```
#LOAD DATA
url = "https://raw.githubusercontent.com/Statology/Python-Guides/main/default.csv"
data=pd.read_csv(url)
print(data.head(10))

#find total observations in dataset
print(len(data.index))
data.info()

#FIT LOGISTIC REGRESSION MODEL
X = data[['student', 'balance','income']]
y = data['default']

#split the dataset into training (70%) and testing (30%) sets
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.3,random_state=0)

#instantiate the model
log_regression = LogisticRegression()

#fit the model using the training data
log_regression.fit(X_train,y_train)

#use model to make predictions on test data
y_pred = log_regression.predict(X_test)

#MODEL DIAGNOSTICS
cnf_matrix = metrics.confusion_matrix(y_test, y_pred)
#cnf_matrix
print("Accuracy:",metrics.accuracy_score(y_test, y_pred))
print("Recall:",metrics.recall_score(y_test, y_pred))
print("Precision:",metrics.precision_score(y_test, y_pred))

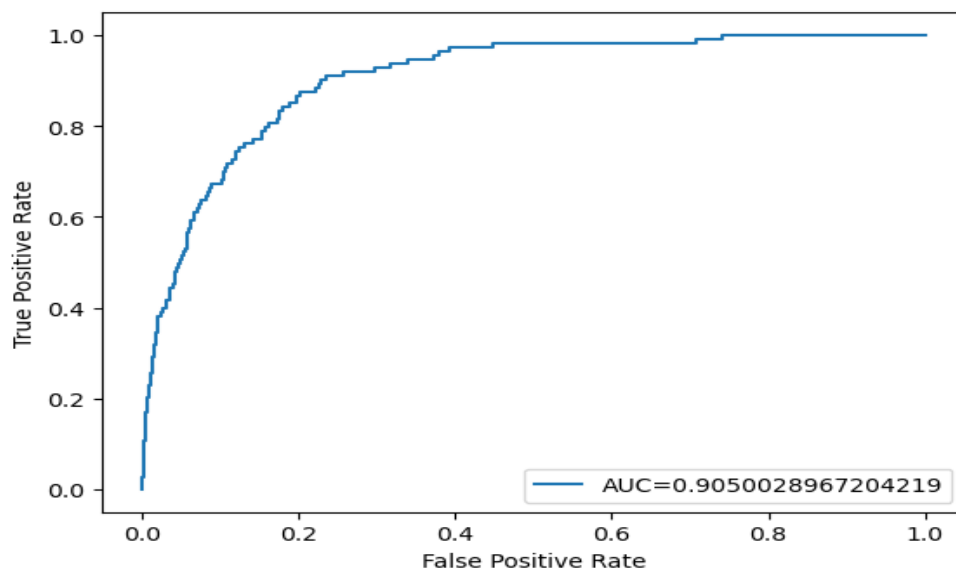
#plot ROC curve
y_pred_proba = log_regression.predict_proba(X_test)[::,1]
fpr, tpr, _ = metrics.roc_curve(y_test, y_pred_proba)
auc = metrics.roc_auc_score(y_test, y_pred_proba)
plt.plot(fpr,tpr,label="AUC="+str(auc))
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc=4)
plt.show()
```

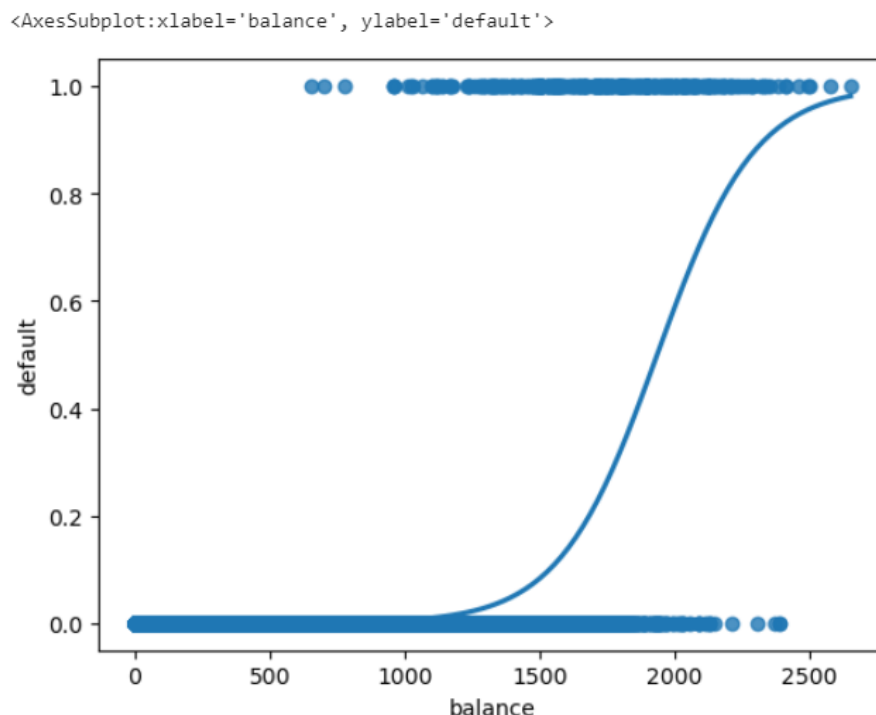
```
#Plot Logistic Regression Curve
x = data['balance']
y = data['default']
sns.regplot(x=x, y=y, data=data, logistic=True, ci=None)
```

OUTPUT

	default	student	balance	income
0	0	0	729.526495	44361.625074
1	0	1	817.180407	12106.134700
2	0	0	1073.549164	31767.138947
3	0	0	529.250605	35704.493935
4	0	0	785.655883	38463.495879
5	0	1	919.588530	7491.558572
6	0	0	825.513331	24905.226578
7	0	1	808.667504	17600.451344
8	0	0	1161.057854	37468.529288
9	0	0	0.000000	29275.268293
10000				

```
Accuracy: 0.9633333333333334
Recall: 0.17699115044247787
Precision: 0.5405405405405406
```





Predict and Predict_proba

The difference between predict and predict_proba is that predict will give you output like 0,1. Whereas predict_proba will give you the probability value of y being 0 or 1.

predict_proba gives you the probabilities for the target (0 and 1 in your case) in array form.

The predict method is used to predict the actual class while predict_proba method can be used to infer the class probabilities

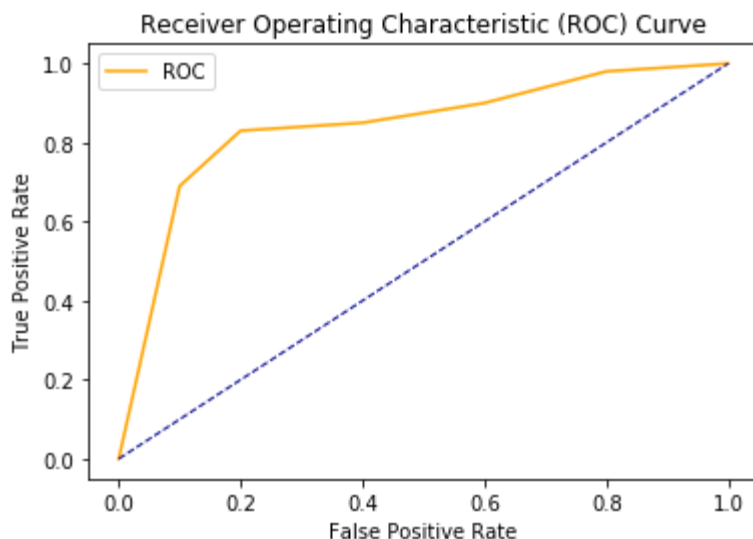
Using [:,1] in the code will give you the probabilities of getting the output as 1. If you replace 1 with 0 in the above code, you will only get the probabilities of getting the output as 0.

Introduction to AUC - ROC Curve

The ROC curve stands for **Receiver Operating Characteristic curve**. ROC curves display the performance of a classification model.

AUC–ROC curve is the model selection metric for bi–multi class classification problem. ROC is a probability curve for different classes. ROC tells us how good the model is for distinguishing the given classes, in terms of the predicted probability.

A typical ROC curve has False Positive Rate (FPR) on the X-axis and True Positive Rate (TPR) on the Y-axis.



The area covered by the curve is the area between the orange line (ROC) and the axis. This area covered is AUC. The bigger the area covered, the better the machine learning models is at distinguishing the given classes. Ideal value for AUC is 1.

Support Vector Machine

Introduction to SVM

What is a Support Vector Machine?

It is a supervised machine learning problem where we try to find a hyperplane that best separates the two classes.

Now the question is which hyperplane does it select? There can be an infinite number of hyperplanes passing through a point and classifying the two classes perfectly. So, which one is the best?

Well, SVM does this by finding the maximum margin between the hyperplanes that means maximum distances between the two classes.

The hyperplane equation dividing the points (for classifying) can now easily be written as:

$$\mathbf{H}: \mathbf{w}^T(\mathbf{x}) + \mathbf{b} = 0$$

$\mathbf{w}$ is a weight vector

$\mathbf{x}$ is input vector

$\mathbf{b}$ is bias

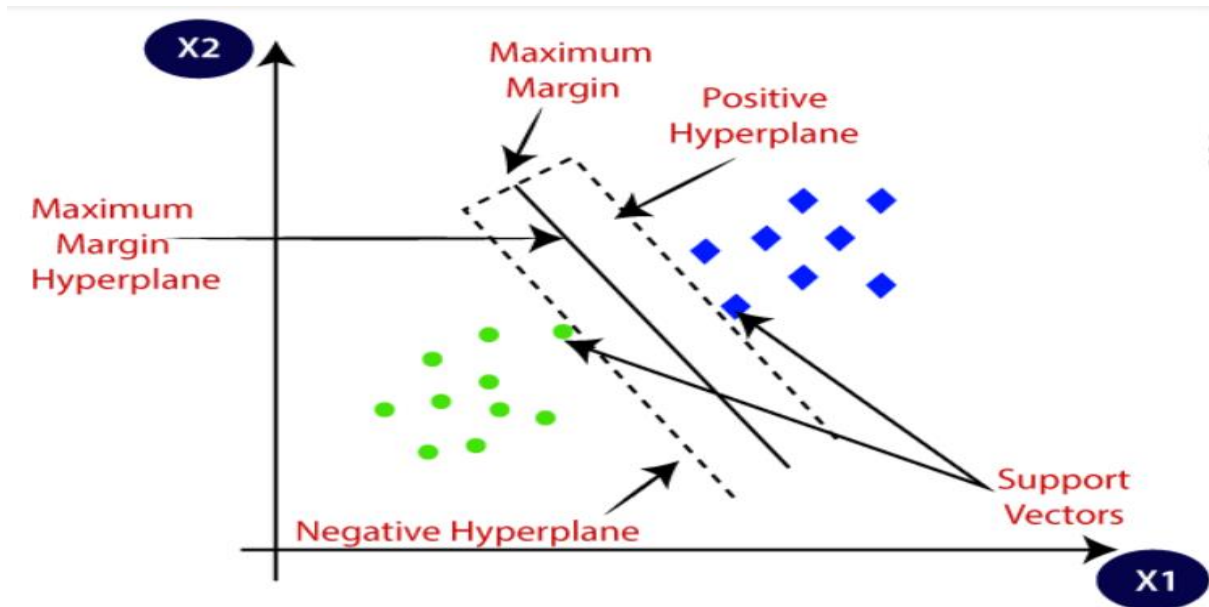


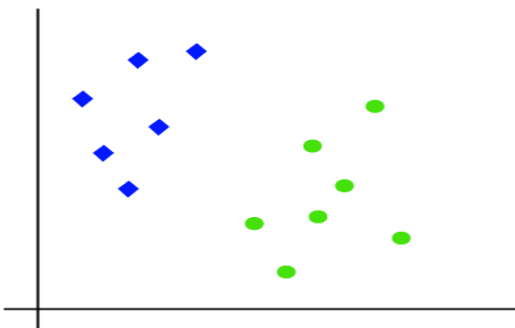
Image 1

Support Vectors: These are the points that are closest to the hyperplane. A separating line will be defined with the help of these data points.

Margin: it is the distance between the hyperplane and the observations closest to the hyperplane (support vectors). In SVM large margin is considered a good margin.

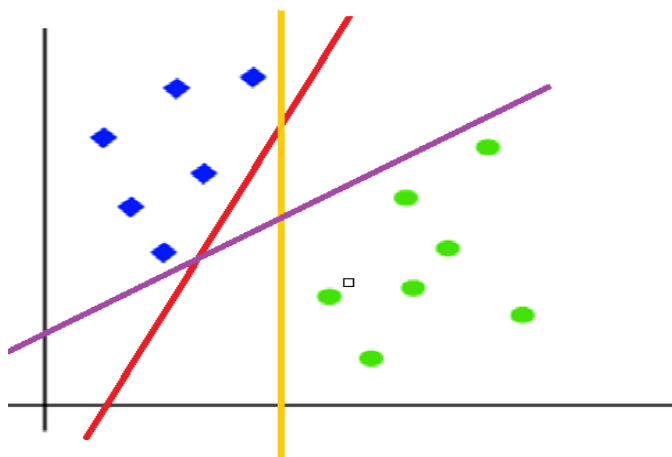
How does Support Vector Machine work?

SVM is defined such that it is defined in terms of the support vectors only, we don't have to worry about other observations since the margin is made using the points which are closest to the hyperplane (support vectors), whereas in logistic regression the classifier is defined over all the points. Hence SVM enjoys some natural speed-ups. Let's understand the working of SVM using an example. Suppose we have a dataset that has two classes (green and blue). We want to classify that the new data point as either blue or green.

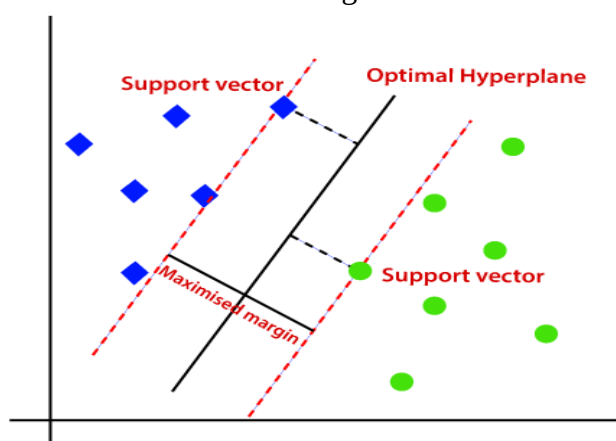


To classify these points, we can have many decision boundaries, but the question is which is the best and how do we find it?

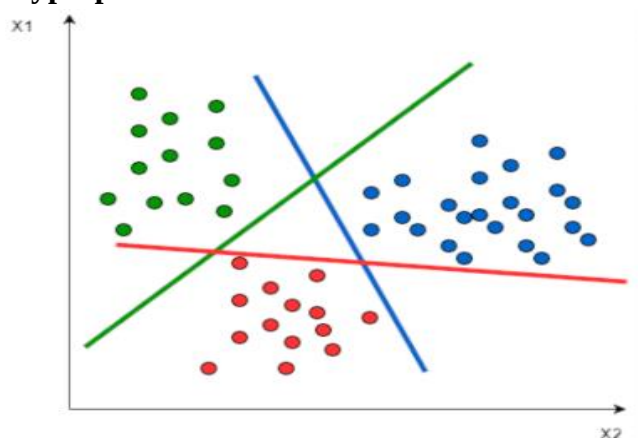
NOTE: Since we are plotting the data points in a 2-dimensional graph we call this decision boundary a **straight line** but if we have more dimensions, we call this decision boundary a **"hyperplane"**



The best hyperplane is that plane that has the maximum distance from both the classes, and this is the main aim of SVM. This is done by finding different hyperplanes which classify the labels in the best way then it will choose the one which is farthest from the data points or the one which has a maximum margin.



Hyperplane for Multiclass Classification



Parameters of SVC() of SVM

C

C is the penalty parameter of the error term. It controls the trade-off between smooth decision boundary and classifying the training points correctly.

Gamma

gamma is a parameter for nonlinear hyperplanes. The higher the gamma value it tries to exactly fit the training data set

Kernel

A kernel is a function used in SVM for **helping to solve problems**. They provide shortcuts to avoid complex calculations. The amazing thing about kernel is that we can go to higher dimensions and perform smooth calculations with the help of it.

Sometimes, we cannot have a hyperplane for certain problems. This problem arises when we go up to higher dimensions and try to form a hyperplane.

A kernel helps to form the hyperplane in the higher dimension without raising the complexity.

Some Kernel Functions are:

Radial Basis Function kernel

Linear kernel

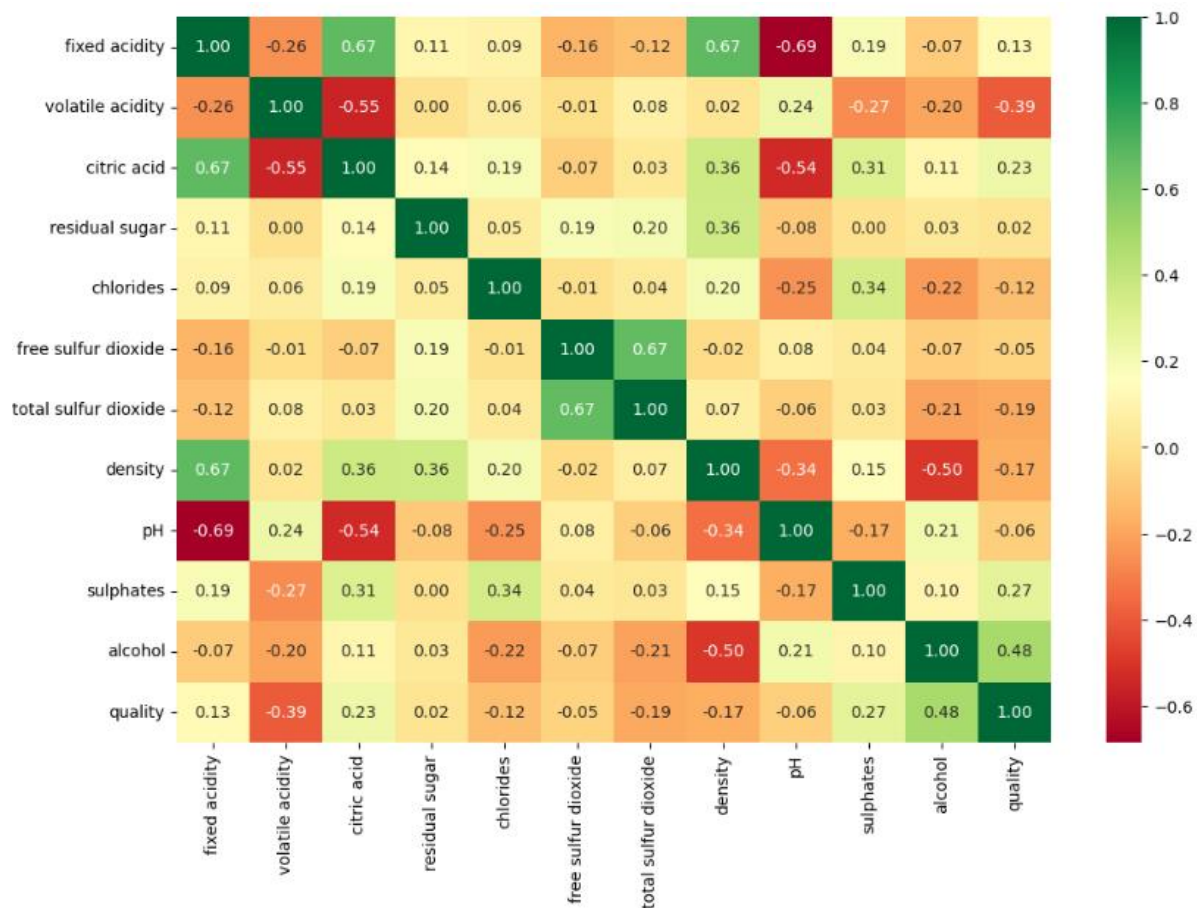
Polynomial kernel

Build SVM Model

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
df=pd.read_csv('C:/Users/Hp/OneDrive/Desktop/Datasets/winequality-red.csv')
df.head(10)
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
5	7.4	0.66	0.00	1.8	0.075	13.0	40.0	0.9978	3.51	0.56	9.4	5
6	7.9	0.60	0.06	1.6	0.069	15.0	59.0	0.9964	3.30	0.46	9.4	5
7	7.3	0.65	0.00	1.2	0.065	15.0	21.0	0.9946	3.39	0.47	10.0	7
8	7.8	0.58	0.02	2.0	0.073	9.0	18.0	0.9968	3.36	0.57	9.5	7
9	7.5	0.50	0.36	6.1	0.071	17.0	102.0	0.9978	3.35	0.80	10.5	5

```
corrmat=df.corr()
f,ax=plt.subplots(figsize=[12,8])
sns.heatmap(corrmat, annot=True, fmt='.2f',cmap='RdYlGn', ax=ax)
plt.show()
```



```

from sklearn.preprocessing import StandardScaler
scaler=StandardScaler()
new_data=scaler.fit_transform(df.drop(labels=['quality'],axis=1))

```

```
new_data
```

```

array([[ -0.52790985,  0.96050573, -1.3943694 , ...,  1.29213992,
        -0.59129553, -0.96331098],
       [ -0.29691763,  1.9654189 , -1.3943694 , ..., -0.73006273,
         0.14487375, -0.58596616],
       [ -0.29691763,  1.29547678, -1.18800687, ..., -0.33866867,
        -0.03916857, -0.58596616],
       ...,
       [ -1.16313844, -0.10023596, -0.72369117, ...,  0.70504883,
         0.57430582,  0.54606832],
       [ -1.39413066,  0.65344892, -0.7752818 , ...,  1.68353398,
         0.32891606, -0.20862133],
       ...])

```

```
x=new_data
```

```
y=df['quality']
```

```
from sklearn.model_selection import train_test_split
```

```
xtrain,xtest,ytrain,ytest=train_test_split(x,y,test_size=0.33,random_state=42)
```

```
from sklearn.svm import SVC
```

```
model=SVC()
```

```
model.fit(xtrain,ytrain)
```

```
model.predict(xtest)
```

```
array([6, 5, 5, 6, 6, 5, 5, 6, 6, 6, 6, 6, 5, 5, 6, 5, 5, 5, 5, 6, 5,
       6, 5, 5, 5, 6, 6, 5, 6, 6, 6, 6, 5, 6, 5, 7, 5, 6, 5, 6, 5, 6, 5,
       6, 5, 5, 5, 5, 6, 6, 6, 5, 5, 5, 5, 6, 5, 5, 6, 5, 5, 7, 6, 5, 5,
       5, 6, 6, 5, 6, 6, 6, 6, 6, 5, 6, 5, 6, 6, 5, 6, 6, 5, 7, 6, 5, 6,
       6, 5, 5, 7, 5, 6, 7, 5, 6, 6, 6, 5, 5, 6, 5, 6, 5, 6, 7, 5, 6, 5,
       5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 6, 6, 5, 6, 5, 6, 5, 6, 6, 5,
       5, 5, 6, 6, 6, 6, 6, 5, 5, 6, 5, 6, 5, 5, 5, 6, 5, 6, 5, 6, 6, 5, 6,
       6, 6, 5, 5, 5, 6, 7, 5, 5, 6, 5, 5, 5, 5, 6, 6, 5, 5, 6, 7, 5, 5,
       5, 6, 6, 5, 6, 7, 5, 6, 5, 5, 6, 6, 6, 6, 6, 7, 5, 5, 5, 5, 7, 5,
       6, 5, 6, 5, 6, 5, 5, 6, 6, 6, 6, 6, 5, 5, 5, 5, 5, 6, 6, 5, 5, 6,
       5, 6, 5, 7, 5, 6, 6, 5, 5, 7, 6, 5, 6, 5, 6, 5, 6, 7, 6, 5, 6, 6,
       5, 5, 6, 5, 6, 7, 5, 6, 7, 5, 5, 6, 5, 6, 6, 5, 5, 5, 6, 5, 5, 5,
       5, 6, 7, 5, 5, 5, 5, 6, 5, 5, 7, 5, 5, 5, 5, 5, 5, 6, 6, 6, 6, 6,
       6, 6, 6, 6, 7, 5, 6, 5, 6, 5, 6, 6, 6, 5, 6, 6, 7, 6, 6, 7, 5, 5,
       5, 5, 6, 5, 6, 7, 6, 7, 6, 5, 6, 5, 7, 5, 6, 6, 7, 7, 6, 5, 6, 6,
       5, 6, 5, 5, 7, 5, 6, 5, 6, 6, 5, 5, 5, 5, 6, 6, 6, 5, 5, 6,
       5, 5, 6, 6, 5, 5, 6, 6, 5, 5, 5, 6, 5, 5, 5, 6, 5, 5, 6, 7,
       6, 6, 6, 5, 6, 5, 5, 6, 6, 5, 7, 5, 6, 5, 6, 6, 6, 6, 6, 5,
       6, 5, 5, 5, 5, 6, 5, 5, 6, 5, 6, 7, 6, 7, 5, 7, 6, 6, 6, 6, 6,
       5, 5, 6, 6, 6, 6, 6, 6, 5, 5, 6, 6, 6, 6, 6, 5, 6, 6, 5, 5, 5,
       5, 6, 6, 6, 6, 6, 6, 6, 5, 6, 6, 6, 5, 5, 6, 7, 5, 5, 5, 6, 6, 6,
       5, 6, 5, 5, 5, 6, 5, 5, 5, 6, 6, 6, 5, 6, 6, 6, 5, 6, 5, 5, 7, 7,
       5, 6, 6, 5, 5, 5, 5, 6, 5, 6, 6, 5, 6, 5, 6, 5, 6, 5, 5, 5],
      dtype=int64)
```

```
from sklearn.metrics import accuracy_score
accuracy_score(ytest,model.predict(xtest))
0.6299810246679317
```

Build SVM model on Iris Dataset

```
import numpy as np
import pandas as pd
```

```
URL = 'https://raw.githubusercontent.com/aviralb13/codes/main/datas/iris.csv'
data = pd.read_csv(URL)
data.head()
```

```
features = ['sepal_length','sepal_width','petal_length','petal_width']
x = data[features]
y = data['species']
```

```
from sklearn.model_selection import train_test_split
train_x,test_x,train_y,test_y = train_test_split(x,y)
```

```
from sklearn.svm import SVC
model = SVC()
model.fit(train_x,train_y)
```

```
prediction = model.predict(test_x)
print(prediction)
from sklearn.metrics import accuracy_score
```

```
accuracy_score(test_y, prediction)
```

Output

```
['Iris-setosa' 'Iris-versicolor' 'Iris-versicolor' 'Iris-virginica'
'Iris-virginica' 'Iris-setosa' 'Iris-virginica' 'Iris-setosa'
'Iris-setosa' 'Iris-setosa' 'Iris-virginica' 'Iris-virginica'
'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-virginica'
'Iris-versicolor' 'Iris-setosa' 'Iris-setosa' 'Iris-virginica'
'Iris-versicolor' 'Iris-virginica' 'Iris-setosa' 'Iris-versicolor'
'Iris-setosa' 'Iris-versicolor' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa'
'Iris-setosa' 'Iris-setosa' 'Iris-versicolor' 'Iris-setosa'
'Iris-versicolor' 'Iris-virginica' 'Iris-virginica' 'Iris-setosa'
'Iris-setosa']
0.9210526315789473
```

Plot Hyperplane on Iris Dataset

```
import matplotlib.pyplot as plt
from sklearn import svm, datasets
from sklearn.inspection import DecisionBoundaryDisplay

import numpy as np
import pandas as pd
# import some data to play with
iris = datasets.load_iris()
print(iris)
# Take the first two features. We could avoid this by using a two-dim dataset
X = iris.data[:, :2]
y = iris.target
C = 1.0 # SVM regularization parameter
models = (
    svm.SVC(kernel="linear", C=C),
    svm.LinearSVC(C=C, max_iter=10000),
    svm.SVC(kernel="rbf", gamma=0.7, C=C),
    svm.SVC(kernel="poly", degree=3, gamma="auto", C=C),
)
models = (clf.fit(X, y) for clf in models)

# title for the plots
titles = (
    "SVC with linear kernel",
    "LinearSVC (linear kernel)",
    "SVC with RBF kernel",
    "SVC with polynomial (degree 3) kernel",
)

# Set-up 2x2 grid for plotting.
```

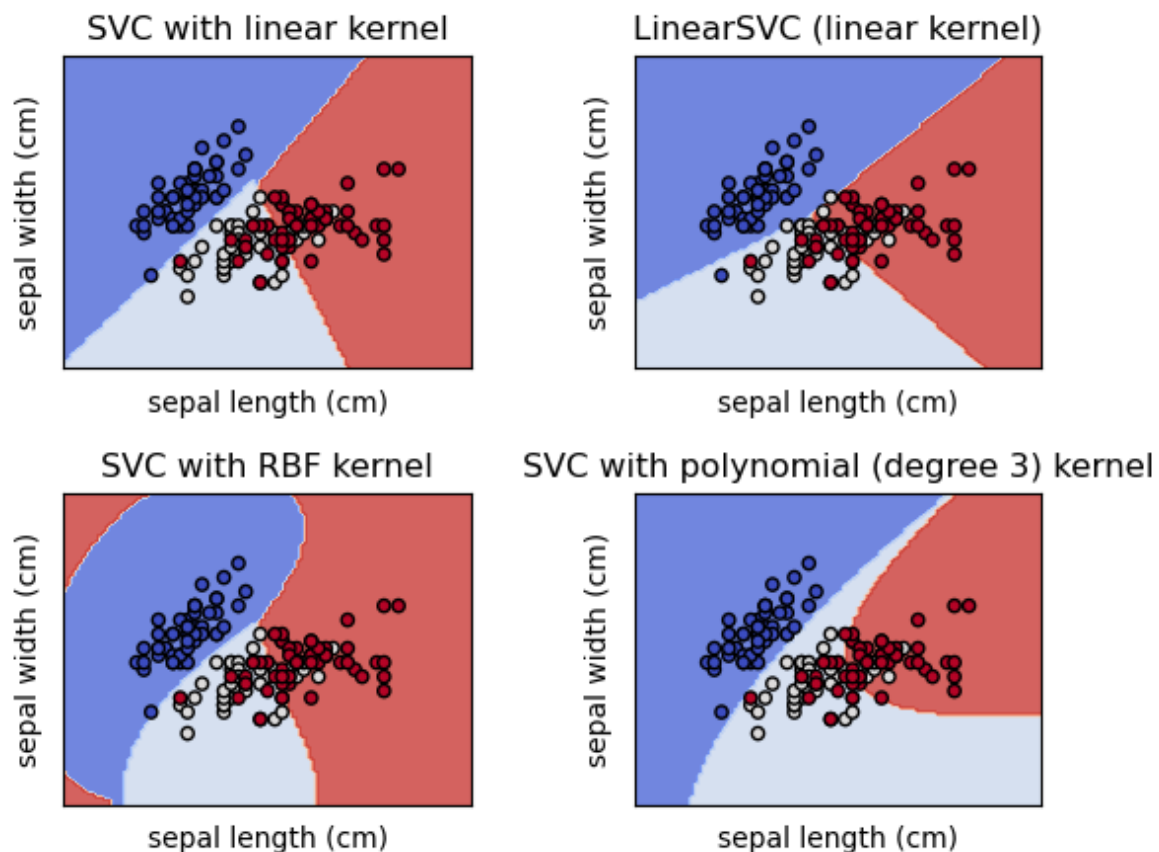
```

fig, sub = plt.subplots(2, 2)
plt.subplots_adjust(wspace=0.4, hspace=0.4)

X0, X1 = X[:, 0], X[:, 1]

for clf, title, ax in zip(models, titles, sub.flatten()):
    disp = DecisionBoundaryDisplay.from_estimator(
        clf,
        X,
        response_method="predict",
        cmap=plt.cm.coolwarm,
        alpha=0.8,
        ax=ax,
        xlabel=iris.feature_names[0],
        ylabel=iris.feature_names[1],
    )
    ax.scatter(X0, X1, c=y, cmap=plt.cm.coolwarm, s=20, edgecolors="k")
    ax.set_xticks(())
    ax.set_yticks(())
    ax.set_title(title)
plt.show()

```



Ensemble Learning

What Is Ensemble?

The ensemble methods in machine learning combine the insights obtained from multiple learning models to facilitate accurate and improved decisions.

ensemble methods in machine learning employ a set of models and take advantage of the blended output, which, compared to a solitary model, will most certainly be a superior option when it comes to prediction accuracy

Basic Ensemble Techniques

Max Voting:

It is mainly used for classification problems. The method consists of building multiple models independently and getting their individual output called 'vote'. The class with maximum votes is returned as output.

For example:

You are asked 5 colleagues to give the review about your movie we will assume three of five rated as 3 and two of them gave 4. Since the majority of the given rating is 4. Then the final rating will be taken as 4.

The result of max voting is:

C-1	C-2	C-3	C-4	C-5	Final rating
3	4	3	4	3	3

Averaging:

The Averaging is similar to the max voting, but in this multiple prediction are done by each data point in averaging. In this method we can take average of all the models and make the final prediction. It is used for both regression and classification problems.

For example:

In the averaging method would take the average of all the values.

i.e. $(3+4+3+4+3)/5 = 3.4$

Weighted Average:

This is the extended version of averaging method in this every model is assigned different weights which defines the importance of each model for the predictions.

For example:

From your friends three friends don't have the much knowledge about this field and others are critics then these two friends are given most important compared to others.

Then the result is,

$[(5*0.23) + (4*0.23) + (5*0.18) + (4*0.18) + (4*0.18)] = 4.41$.

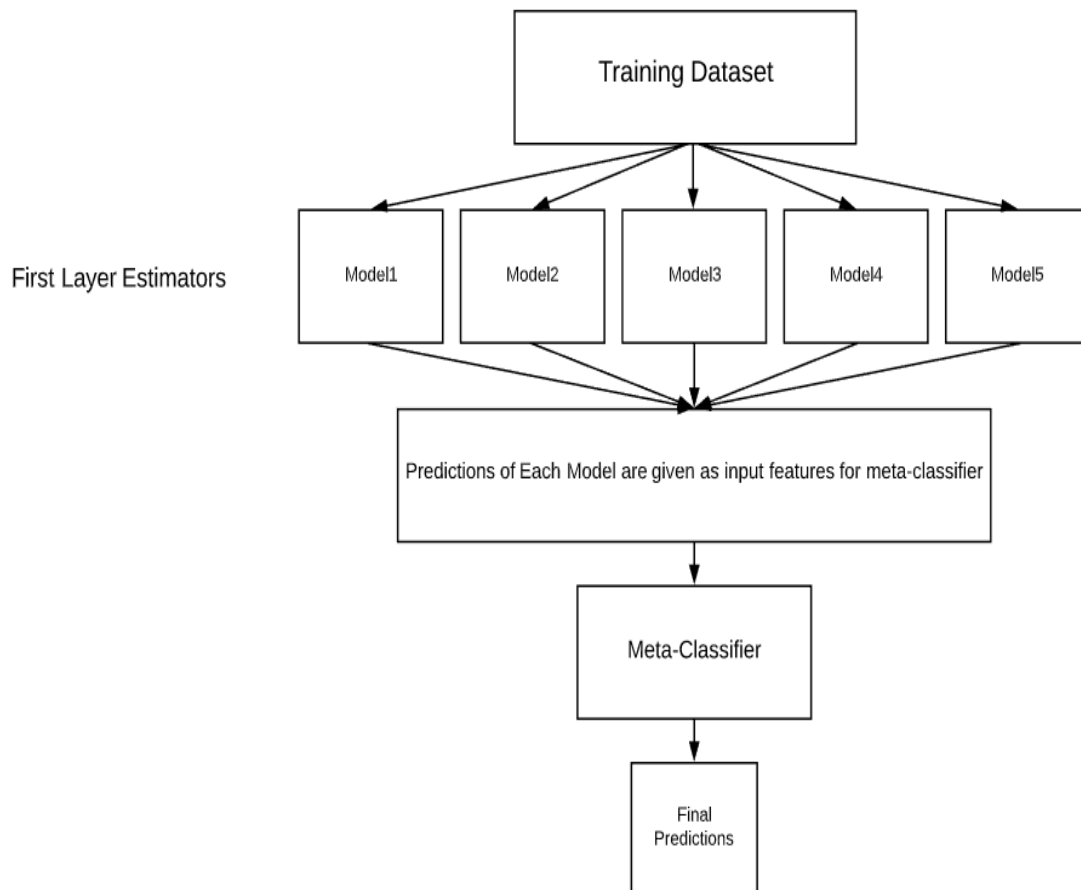
C-1	C-2	C-3	C-4	C-5	Final rating
weight	0.23	0.23	0.18	0.18	0.18
rating	5	4	5	4	4
					4.41

Advanced Ensemble techniques:

Stacking:

Stacking is a way of ensembling classification or regression models it consists of two-layer estimators. The first layer consists of all the baseline models that are used to predict the outputs on the test datasets. The second layer consists of Meta-Classifier or Regressor which takes all the predictions of baseline models as an input and generate new predictions.

Stacking Architecture:



Blending:

This blending technique is very similar to the stacking but it uses only validation set from the training set. In other words the predictions are mainly done on the validation set and the validation set and predictions are used to build a model and that model will run on test set.

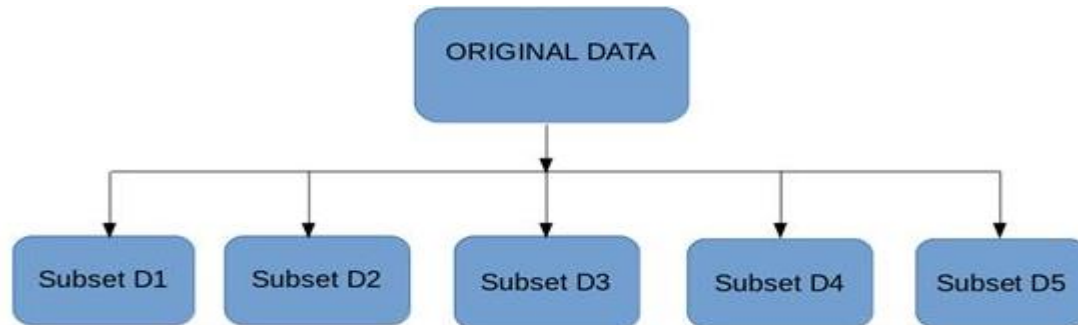
The steps for blending ensemble technique are as follows:

- First, the training data is divided into training and validation sets.
- Base models are fit on the training set.
- The predictions are made on the validation set and the test set.
- Predictions on validation data and validation data are used as features to build a final model.

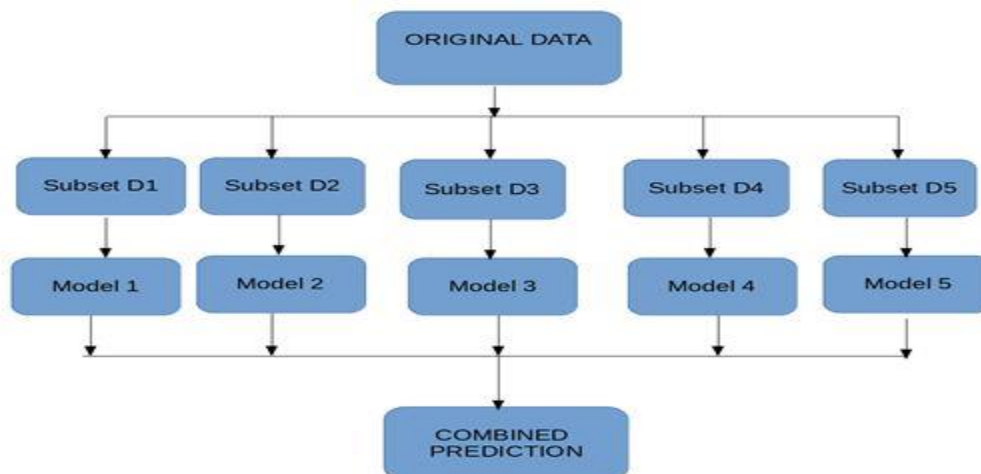
- This model is used to make final predictions on the test data.

Bagging:

Bagging or bootstrap aggregation is an ensemble technique that uses the sample datasets. These sample datasets are none other than the part of or the subset of original dataset.

**In this bagging**

1. We can create a multiple subsets from original dataset.
2. Create a base model on each subset.
3. These all models are independent with each other and run in parallel.
4. Combine all these models to get the final predictions.

**Boosting:**

Boosting is a sequential process, each subsequent model correct the predictions of previous model and the subsequent models are dependent on the previous models.

Basically, the boosting algorithm is done like this

1. We create a subset from the original dataset.
2. All data points are given equal weights.
3. Create the base model for the subset
4. The model is used to make the predictions on the total data set.
5. Now errors are calculated by using the actual and predicted values.
6. If you find any in corrected prediction, that gets higher weights.
7. For that misclassified we can create another model to reduce that error.
8. And now create multiple models to reduce the error in the previous model.

9. And the final model is combining all the weak models to get the weighted mean of all models.

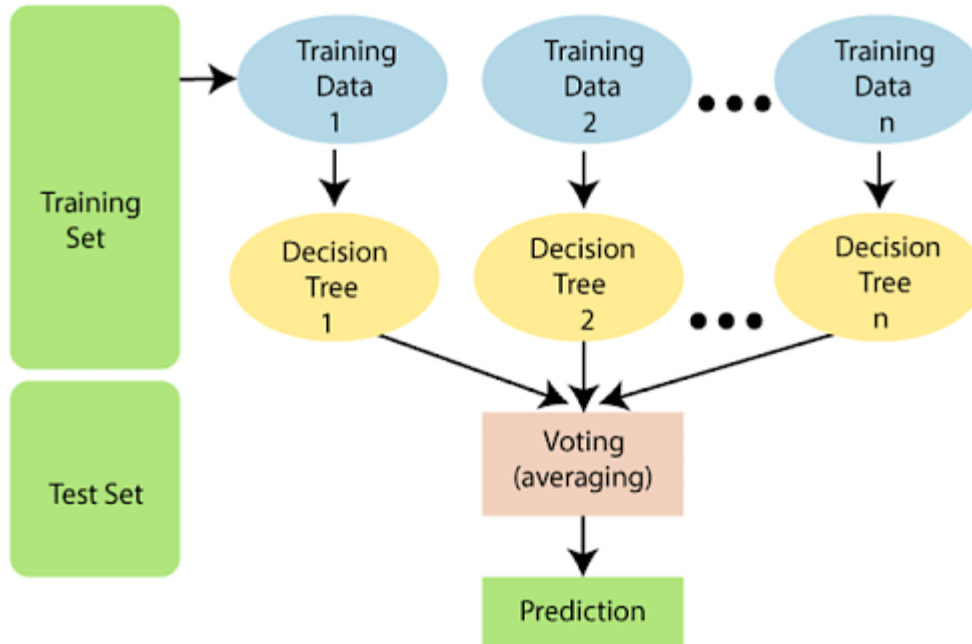
10. Individual learners are not performing well on the entire dataset they perform well only on some part of the dataset that's why we can use the Boosting algorithm to combine all weak learners and form a strong learner.

Random Forest Algorithm

Introduction

- A Random Forest Algorithm is a supervised machine learning algorithm which is extremely popular and is used for Classification and Regression problems in Machine Learning.
- We know that a forest comprises numerous trees, and the more trees more it will be robust. Similarly, the greater the number of trees in a Random Forest Algorithm, the higher its accuracy and problem-solving ability.
- Random Forest is a classifier that contains several decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset.
- It is based on the concept of ensemble learning which is a process of combining multiple classifiers to solve a complex problem and improve the performance of the model.

How does it work?



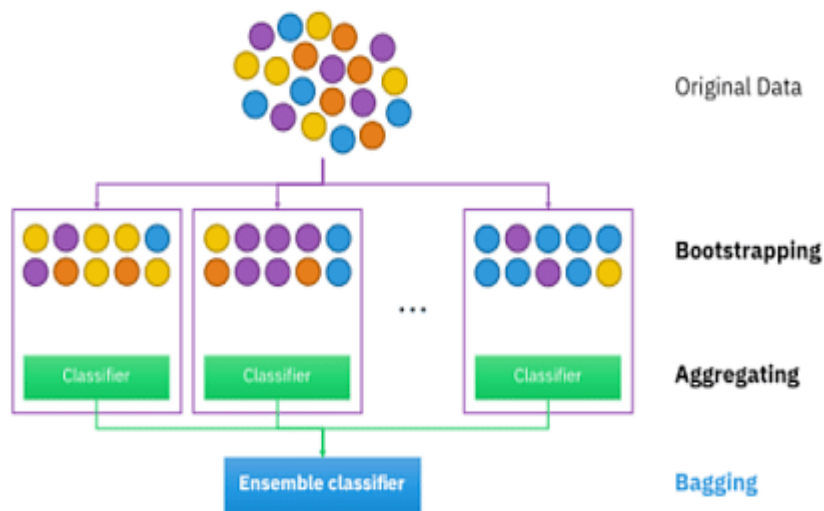
The following steps explain the working Random Forest Algorithm:

Step 1: Select random samples from a given data or training set.

Step 2: This algorithm will construct a decision tree for every training data.

Step 3: Voting will take place by averaging the decision tree.

Step 4: Finally, select the most voted prediction result as the final prediction result.



Why Use a Random Forest Algorithm?

There are a lot of benefits to using Random Forest Algorithm, but one of the main advantages is that it **reduces the risk of overfitting** and the **required training time**. Additionally, it offers **a high level of accuracy**. Random Forest algorithm **runs efficiently in large databases** with higher dimensions.

Important Hyperparameters

Hyperparameters are used in random forests to either enhance the performance and predictive power of models or to make the model faster.

The following hyperparameters are used to enhance the predictive power:

n_estimators: int, default=100 Number of trees built by the algorithm before averaging the product

criterion{"gini", "entropy", "log_loss"}, default="gini": The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "log_loss" and "entropy"

max_features: Maximum number of features random forest uses before considering splitting a node.

mini_sample_leaf: Determines the minimum number of leaves required to split an internal node.

Application of Random Forest

Some major Applications of Random Forest in different sectors:

- Banking Industry
 - Credit Card Fraud Detection
 - Customer Segmentation
 - Predicting Loan Defaults on LendingClub.com
- Healthcare and Medicine
 - Cardiovascular Disease Prediction
 - Diabetes Prediction
 - Breast Cancer Prediction
- Stock Market
 - Stock Market Prediction
 - Stock Market Sentiment Analysis
 - Bitcoin Price Detection
- E-Commerce
 - Product Recommendation
 - Price Optimization

Advantages of Random forest

The algorithm has the following advantages:-

- It can solve both regression and classification problems.
- It helps us to deal with the overfitting issue by taking the average of each decision tree.
- It can easily handle large datasets.

Build Random Forest-based model in python for Breast Cancer Wisconsin (diagnostic) dataset from sci-kit learn Or dataset from UCI , Kaggle

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
#load dataset
```

```
data = pd.read_csv("C:/Users/Hp/OneDrive/Desktop/Datasets/breastcancer.csv")
```

```
# print top 5 rows of the data
data.head()
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	...	texture_worst	perimeter_worst	ar
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	...	17.33	184.60	
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	...	23.41	158.80	
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	...	25.53	152.50	
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	...	26.50	98.87	
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	...	16.67	152.20	

5 rows × 33 columns

```
data = data.drop("Unnamed: 32",axis=1)
data['diagnosis'] = data['diagnosis'].map({'M':1,'B':0})
X=data.drop("diagnosis",axis=1)
y = data["diagnosis"]

from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.20,random_state=21)
from sklearn.ensemble import RandomForestClassifier
# building classifier
rfc = RandomForestClassifier(n_estimators=110)
# training the model on the training set
rfc.fit(X_train,y_train)

# prediction on test data
y_pred = rfc.predict(X_test)

# checking the accuracy of the model using actual and predicted values
from sklearn.metrics import accuracy_score, classification_report
print(accuracy_score(y_test,y_pred)*100)
97.36842105263158
```

```
y_pred
array([0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1,
       0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 1,
       0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0,
       1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1,
       1, 0, 0, 1], dtype=int64)
```

```
print(classification_report(y_test,y_pred))
```

	precision	recall	f1-score	support
0	0.97	0.97	0.97	75
1	0.95	0.95	0.95	39
accuracy			0.96	114
macro avg	0.96	0.96	0.96	114
weighted avg	0.96	0.96	0.96	114